

COMP442 Assignment 2

Charles-Philippe La Richeliere

February 2024

Grammar

This is the final LL(1) grammar, in the university of calgary syntax.

START -> PROG eof .

ADDOP -> plus
 | minus
 | or .

APARAMS -> EXPR REPTAPARAMS1
 | .

APARAMSTAIL -> comma EXPR .

ARITHEXPR -> TERM RIGHTRECARITHEXPR .

ARRAYSIZE -> lsqbr Flsqbr .

Flsqbr -> intnum rsqbr
 | rsqbr .

ASSIGNOP -> equal .

EXPR -> ARITHEXPR OPTRELEXPR .

OPTRELEXPR -> RELOP ARITHEXPR | .

FACTOR -> id FACTOR2 REPTVARIABLEORFUNCTIONCALL
 | intlit
 | floatlit
 | lpar ARITHEXPR rpar
 | not FACTOR
 | SIGN FACTOR .

```

FACTOR2 -> lpar APARAMS rpar | REPTIDNEST1 .

FPARAMS -> id colon TYPE REPTFPARAMS3 REPTFPARAMS4
          |.

FPARAMSTAIL -> comma id colon TYPE REPTFPARAMSTAIL4 .

FUNCBODY -> lcurbr REPTFUNCBODY1 rcurbr .

FUNCDECL -> FUNCHEAD semi .

FUNCDEF -> FUNCHEAD FUNCBODY .

FUNCHEAD -> func id lpar FPARAMS rpar arrow RETURNTYPE .

REPTVARIABLEORFUNCTIONCALL -> IDNEST REPTVARIABLEORFUNCTIONCALL | .
IDNEST -> dot id IDNEST2 .
IDNEST2 -> lpar APARAMS rpar | REPTIDNEST1 .

IMPLDEF -> impl id lcurbr REPTIMPLDEF3 rcurbr .

INDICE -> lsqbr ARITHEXPR rsqbr .

MEMBERDECL -> FUNCDECL
              |VARDECL .

MULTOP -> mult
          |div
          |and .

OPTSTRUCTDECL2 -> inherits id REPTOPTSTRUCTDECL22
                  |.

PROG -> REPTPROG0 .

RELEXPR -> ARITHEXPR RELOP ARITHEXPR .

RELOP -> eq
          |neq
          |lt
          |gt
          |leq
          |geq .

REPTAPARAMS1 -> APARAMSTAIL REPTAPARAMS1

```

```

        | .

REPTFPARAMS3 -> ARRAYSIZE REPTFPARAMS3
        | .

REPTFPARAMS4 -> FPARAMSTAIL REPTFPARAMS4
        | .

REPTFPARAMSTAIL4 -> ARRAYSIZE REPTFPARAMSTAIL4
        | .

REPTFUNCBODY1 -> VARDECLORSTAT REPTFUNCBODY1
        | .

REPTIDNEST1 -> INDICE REPTIDNEST1
        | .

REPTIMPLDEF3 -> FUNCDEF REPTIMPLDEF3
        | .

REPTOPTSTRUCTDECL22 -> comma id REPTOPTSTRUCTDECL22
        | .

REPTPROG0 -> STRUCTORIMPLORFUNC REPTPROG0
        | .

REPTSTATBLOCK1 -> STATEMENT REPTSTATBLOCK1
        | .

REPTSTRUCTDECL4 -> VISIBILITY MEMBERDECL REPTSTRUCTDECL4
        | .

REPTVARDECL4 -> ARRAYSIZE REPTVARDECL4
        | .

RETURNTYPE -> TYPE
        | void .

RIGHTRECARITHEXPR ->
        | ADDOP TERM RIGHTRECARITHEXPR .

RIGHTRECTERM ->
        | MULTOP FACTOR RIGHTRECTERM .

SIGN -> plus
        | minus .

```

```

STATBLOCK -> lcurbr REPTSTATBLOCK1 rcurbr
            |STATEMENT
            |.

STATEMENT -> id STATEMENTBODY semi
            |if lpar RELEXPR rpar then STATBLOCK else STATBLOCK semi
            |while lpar RELEXPR rpar STATBLOCK semi
            |read lpar VARIABLE rpar semi
            |write lpar EXPR rpar semi
            |return lpar EXPR rpar semi .

STATEMENTBODY -> REPTIDNEST1 CHECKFORENDVAR
                | lpar APARAMS rpar CHECKFOREENDFUNC .

VARFUNCSTATENEST -> dot id VARFUNCSTATENEST2 .

VARFUNCSTATENEST2 -> lpar APARAMS rpar CHECKFOREENDFUNC
                    | REPTIDNEST1 CHECKFORENDVAR .

CHECKFOREENDFUNC ->
                  | VARFUNCSTATENEST .

CHECKFORENDVAR -> ASSIGNOP EXPR
                  | VARFUNCSTATENEST .

STRUCTDECL -> struct id OPTSTRUCTDECL2 lcurbr REPTSTRUCTDECL4 rcurbr semi .

STRUCTORIMPLORFUNC -> STRUCTDECL
                    |IMPLDEF
                    |FUNCDEF .

TERM -> FACTOR RIGHTRECTERM .

TYPE -> integer
      |float
      |id .

VARDECL -> let id colon TYPE REPTVARDECL4 semi .

VARDECLORSTAT -> VARDECL
                |STATEMENT .

VARIABLE -> id VARIABLE2 .

VARIABLE2 -> REPTIDNEST1 REPTVARIABLE .
VARIABLE2 -> lpar APARAMS rpar VARIDNEST .

```

```
REPTVARIABLE -> VARIDNEST REPTVARIABLE .
REPTVARIABLE -> .
```

```
VARIDNEST -> dot id VARIDNEST2 .
```

```
VARIDNEST2 -> lpar APARAMS rpar VARIDNEST .
VARIDNEST2 -> REPTIDNEST1 .
```

```
VISIBILITY -> public
              |private .
```

FIRST and FOLLOW sets

These are the first and follow sets for all of the non-terminal symbols in the language:

```
const char *first_start[] = {"struct", "impl", "func", "eof"};
const char *follow_start[] = {};
const char *first_prog[] = {"struct", "impl", "func", "EPSILON"};
const char *follow_prog[] = {"eof"};
const char *first_reptprog0[] = {"struct", "impl", "func", "EPSILON"};
const char *follow_reptprog0[] = {"eof"};
const char *first_structorimplorfunc[] = {"struct", "impl", "func"};
const char *follow_structorimplorfunc[] = {"struct", "impl", "func", "eof"};
const char *first_structdecl[] = {"struct"};
const char *follow_structdecl[] = {"struct", "impl", "func", "eof"};
const char *first_optstructdecl2[] = {"inherits", "EPSILON"};
const char *follow_optstructdecl2[] = {"lcubr"};
const char *first_reptstructdecl4[] = {
    "public",
    "private",
    "EPSILON",
};

const char *follow_reptstructdecl4[] = {"rcubr"};
const char *first_visibility[] = {"private", "public"};
const char *follow_visibility[] = {"let", "func"};
const char *first_memberdecl[] = {"let", "func"};
const char *follow_memberdecl[] = {"rcubr", "public", "private"};
const char *first_funcdecl[] = {"func"};
```

```

const char *follow_funcdecl[] = {"rcubr", "public", "private"};
const char *first_vardecl[] = {"let"};
const char *follow_vardecl[] = {"public", "private", "let", "id", "while",
                                "read", "write", "return", "rcubr"};

const char *first_funchead[] = {"func"};
const char *follow_funchead[] = {"semi", "lcubr"};
const char *first_reptvardecl4[] = {"lsqbr", "EPSILON"};
const char *follow_reptvardecl4[] = {"semi"};
const char *first_fparams[] = {"id", "EPSILON"};
const char *follow_fparams[] = {"rpar"};

const char *first_reptfparams3[] = {"lsqbr", "EPSILON"};
const char *follow_reptfparams3[] = {"rpar", "comma"};
const char *first_returntype[] = {"integer", "float", "void", "id"};
const char *follow_returntype[] = {"semi", "lcubr"};

const char *first_type[] = {"integer", "float", "id"};
const char *follow_type[] = {"rpar", "lcubr", "comma", "lsqbr", "semi"};

const char *first_reptfparams4[] = {"comma", "EPSILON"};
const char *follow_reptfparams4[] = {"rpar"};

const char *first_reptfparamstail4[] = {"lsqbr", "EPSILON"};
const char *follow_reptfparamstail4[] = {"comma", "rpar"};
const char *first_fparamstail[] = {"comma"};
const char *follow_fparamstail[] = {"comma", "rpar"};
const char *first_funcdef[] = {"func"};
const char *follow_funcdef[] = {"struct", "impl", "func", "rcubr", "eof"};
const char *first_reptfuncbody1[] = {"let", "id", "if", "while",
                                      "read", "write", "return", "EPSILON"};
const char *follow_reptfuncbody1[] = {"rcubr"};

const char *first_funcbody[] = {"lcubr"};
const char *follow_funcbody[] = {"struct", "impl", "func", "rcubr", "eof"};
const char *first_vardeclorstat[] = {"let", "id", "if", "while",
                                      "read", "write", "return"};
const char *follow_vardeclorstat[] = {"let", "id", "if", "while",
                                      "read", "write", "return", "rcubr"};
const char *first_statementbody[] = {"lpar", "lsqbr", "assign", "dot"};
const char *follow_statementbody[] = {"else", "semi", "let", "id",
                                      "if", "while", "read", "write",
                                      "return", "rcubr"};
const char *first_statement[] = {"id", "if", "while",
                                  "read", "write", "return"};
const char *follow_statement[] = {"else", "semi", "let", "id", "if",
                                  "while", "read", "write", "return", "rcubr"};

```

```

const char *first_checkforendfunc[] = {"semi", "dot"};
const char *follow_checkforendfunc[] = {"else", "semi", "let", "id",
                                         "if", "while", "read", "write",
                                         "return", "rcubr"};
const char *first_checkforendvar[] = {"assign", "dot"};
const char *follow_checkforendvar[] = {
    "else", "if", "let", "id", "while", "read", "write", "return", "rcubr"};
const char *first_expr[] = {"id", "intnum", "floatnum", "lpar",
                             "not", "plus", "minus"};
const char *follow_expr[] = {"comma", "rpar", "semi"};
const char *first_arithexpr[] = {"id", "intnum", "floatnum", "lpar",
                                  "not", "plus", "minus"};
const char *follow_arithexpr[] = {"rsqbr", "eq", "neq", "lt", "gt",
                                   "leq", "geq", "comma", "rpar", "semi"};
const char *first_term[] = {"id", "intnum", "floatnum", "lpar",
                             "not", "plus", "minus"};
const char *follow_term[] = {"rsqbr", "eq", "neq", "lt", "gt",
                              "leq", "geq", "plus", "minus", "or",
                              "comma", "rpar", "semi"};

const char *first_factor[] = {"id", "intnum", "floatnum", "lpar",
                              "not", "plus", "minus"};
const char *follow_factor[] = {
    "mult", "div", "and", "rsqbr", "eq", "neq", "lt", "gt",
    "leq", "geq", "plus", "minus", "or", "comma", "rpar", "semi"};
const char *first_varfuncstatenest2[] = {"lpar", "lsqbr", "assign", "dot"};
const char *follow_varfuncstatenest2[] = {"else", "semi", "let", "id",
                                           "if", "while", "read", "write",
                                           "return", "rcubr"};
const char *first_varfuncstatenest[] = {"dot"};
const char *follow_varfuncstatenest[] = {"else", "semi", "let", "id",
                                           "if", "while", "read", "write",
                                           "return", "rcubr"};
const char *first_addop[] = {"plus", "minus", "or"};
const char *follow_addop[] = {"intnum", "floatnum", "id", "lpar",
                              "not", "plus", "minus"};
const char *first_indice[] = {"lsqbr"};
const char *follow_indice[] = {"assign", "mult", "div", "and", "lsqbr",
                              "dot", "rsqbr", "eq", "neq", "lt",
                              "gt", "leq", "geq", "plus", "minus",
                              "or", "comma", "rpar", "semi"};
const char *first_arraysize[] = {"lsqbr"};
const char *follow_arraysize[] = {"semi", "lsqbr", "rpar", "comma"};
const char *first_reptoptstructdecl22[] = {"comma", "EPSILON"};
const char *follow_reptoptstructdecl22[] = {"lcubr"};
const char *first_reptvariable[] = {"dot", "EPSILON"};

```

```

const char *follow_reptvariable[] = {"rpar"};
const char *first_varidnest[] = {"dot"};
const char *follow_varidnest[] = {"dot", "rpar"};
const char *first_varidnest2[] = {"lpar", "lsqbr", "EPSILON"};
const char *follow_varidnest2[] = {"dot", "rpar"};
const char *first_rightrecrearithexpr[] = {"plus", "minus", "or", "EPSILON"};
const char *follow_rightrecrearithexpr[] = {
    "rsqbr", "eq", "neq", "lt", "gt", "leq", "geq", "comma", "rpar", "semi"};
const char *first_variable2[] = {"rpar", "lsqbr", "dot", "EPSILON"};
const char *follow_variable2[] = {"rpar"};
const char *first_relexpr[] = {"id", "intnum", "floatnum", "lpar",
    "not", "plus", "minus"};
const char *follow_relexpr[] = {"rpar"};
const char *first_reptstatblock1[] = {"id", "if", "while", "read",
    "write", "return", "EPSILON"};
const char *follow_reptstatblock1[] = {"rcubr"};

const char *first_reptimpldef3[] = {"func", "EPSILON"};
const char *follow_reptimpldef3[] = {"rcubr"};

const char *first_impldef[] = {"impl"};
const char *follow_impldef[] = {"struct", "impl", "func", "eof"};
const char *first_statblock[] = {"lcubr", "id", "if", "while",
    "read", "write", "return", "EPSILON"};
const char *follow_statblock[] = {"else", "semi"};
const char *first_aparamstail[] = {"comma"};
const char *follow_aparamstail[] = {"comma", "rpar"};
const char *first_rightrecterm[] = {"mult", "div", "and", "EPSILON"};
const char *follow_rightrecterm[] = {"rsqbr", "eq", "neq", "lt", "gt",
    "leq", "geq", "plus", "minus", "or",
    "comma", "rpar", "semi"};
const char *first_relop[] = {"eq", "neq", "lt", "gt", "leq", "geq"};
const char *follow_relop[] = {"intnum", "floatnum", "lpar",
    "not", "plus", "minus"};
const char *first_flsqbr[] = {"intnum", "rsqbr"};
const char *follow_flsqbr[] = {"semi", "lsqbr", "rpar", "comma"};

const char *first_reptaparams1[] = {"comma", "EPSILON"};
const char *follow_reptaparams1[] = {"rpar"};

const char *first_sign[] = {"plus", "minus"};
const char *follow_sign[] = {"id", "intnum", "floatnum", "lpar",
    "not", "plus", "minus"};
const char *first_variable[] = {"id"};
const char *follow_variable[] = {"rpar"};
const char *first_multop[] = {"mult", "div", "and"};

```



```

const char *follow_multop[] = {"id", "intnum", "floatnum", "lpar",
                               "not", "plus", "minus"};

const char *first_assignop[] = {"assign"};
const char *follow_assignop[] = {"id", "floatnum", "intnum", "lpar",
                                  "not", "plus", "minus"};
const char *first_aparams[] = {"id", "intnum", "floatnum", "lpar",
                                "not", "plus", "minus", "EPSILON"};
const char *follow_aparams[] = {"rpar"};
const char *first_idnest2[] = {"EPSILON", "lpar", "lsqbr"};
const char *follow_idnest2[] = {
    "mult", "div", "and", "dot", "rsqbr", "eq", "neq", "lt", "gt",
    "leq", "geq", "plus", "minus", "or", "comma", "rpar", "semi"};
const char *first_idnest[] = {"dot"};
const char *follow_idnest[] = {"mult", "div", "and", "dot", "rsqbr", "eq",
                                "neq", "lt", "gt", "leq", "geq", "plus",
                                "minus", "or", "comma", "rpar", "semi"};
const char *first_reptvariableorfunctioncall[] = {"dot", "EPSILON"};
const char *follow_reptvariableorfunctioncall[] = {
    "mult", "div", "and", "rsqbr", "eq", "neq", "lt", "gt",
    "leq", "geq", "plus", "minus", "or", "comma", "rpar", "semi"};
const char *first_reptidnest1[] = {"lsqbr", "EPSILON"};
const char *follow_reptidnest1[] = {
    "assign", "mult", "div", "and", "dot", "rsqbr", "eq", "neq", "lt",
    "gt", "leq", "geq", "plus", "minus", "or", "comma", "rpar", "semi"};
const char *first_factor2[] = {"lpar", "lsqbr", "EPSILON"};
const char *follow_factor2[] = {
    "mult", "div", "and", "dot", "rsqbr", "eq", "neq", "lt", "gt",
    "leq", "geq", "plus", "minus", "or", "comma", "rpar", "semi"};
const char *first_optrelexpr[] = {"eq", "neq", "lt", "gt",
                                   "leq", "geq", "EPSILON"};
const char *follow_optrelexpr[] = {"comma", "rpar", "semi"};

```

Note that we include epsilon in the FIRST set of all nullable non-terminals, this is for the purpose of our top-down recursive parsing algorithm.

This also the format that we used to store the first and follow sets in the code for the parser.

The FIRST and FOLLOW sets were obtained using the ucalgary grammar tool.

Design

The code is in two files for the parser; `parser.c` and `parsedriver.c` with corresponding header files. There are two additional files for the purposes of testing; `negativetesting.src` and `positivetesting.src`. The makefile has a rule that runs the parser on all files in the test directory, to test the functionality of the parser enter the command: `make testparser`.

The parser is built using the top-down recursive descent method. We store the FIRST and FOLLOW sets obtained from the University of Calgary grammar tool in arrays of strings. We create a function `skip_errors()` that will handle the error handling and recovery (it is implemented as described in the slides for recursive-descent error handling).

We have a `match()` function that matches a terminal symbol and advances the lookahead token, we also have an `in()` function that tests if the current lookahead is in the current non-terminal symbols first or follow set. Note that we do not use the short-circuit (`&&`) evaluation version of the AND operator we use the logical AND operator (`&`), if we use the short circuit evaluation version of the AND operator we get the problem that when we want to pop a function off of the function call sack we inadvertently end the program abruptly which is not the desired behaviour for the parser.

The test files provided are `positivetest.src` and `negativetest.src`. The file `positivetest.src` contains syntactically valid code, that should pass all tests thus the syntax error file should be empty and the derivation file should provide the complete derivation with no non-terminals left in the final sentential form. The file `negativetest.src` contains invalid syntax and these should fail, a partial derivation should be provided in the derivation file and the error file should contain all of the relevant syntax errors. The file `checklist.txt` was used to test the parser functionality extensively.

There are also included in the `/tests` directory files with the `".rules"` suffix these contain the rightmost derivation shown only by the rules that are being applied at each step, one is produced for each test file.

Tools

The main tool that was used for refactoring the grammar is the University of Calgary online grammar tool (<https://smlweb.cpsc.ucalgary.ca/start.html>). The simple syntax and easy availability through a web browser were the main reasons that this tool was used. The tool was used to locate and refactor the ambiguities in the grammar to make the grammar become LL(1). This was done by using the main tool on the website that allows you to enter a browser with the correct notation and it returns the first and follow sets of the grammar along with other relevant information. The tool was used mainly to locate and fix the FIRST and FOLLOW set clashes and ambiguities as well as obtaining the first and follow sets.

A link to the tool with the final grammar entered on the ucalgary grammar tool is available with the following link:

```
https://smlweb.cpsc.ucalgary.ca/vital-stats.php?grammar=START+-%
3E+PROG+eof++.++%OD%OA%OD%OA%OD%OA%OD%OAADDOP+-%3E+plus++%OD%OA+++
+++++%7Cminus+%OD%OA++++++%7COr+. %OD%OA%OD%OAAPARAMS+-%3E+EXPR+REPTAPARAMS1+
%OD%OA++++++%7C. %OD%OA%OD%OAAPARAMSTAIL+-%3E+comma+EXPR+. %OD%
OA%OD%OAARITHEXPR+-%3E+TERM+RIGHTRECARITHEXPR+. %OD%OA%OD%OAARRAYSIZE+
-%3E+lsqbr+Flsqbr+. %OD%OA%OD%OAFlsqbr+-%3E+intnum+rsqbr++%OD%OA+++
+++++%7Crsqbr+. %OD%OA%OD%OAASSIGNOP+-%3E+equal+. %OD%OA%OD%OA%OD%
OAEXPR+-%3E+ARITHEXPR+OPTRELEXPR+. %OD%OA%OD%OAOPTREREXPR+-%3E+RELOP+
ARITHEXPR+%7C+. ++%OD%OA%OD%OAFACTOR+-%3E+id+FACTOR2+REPTVARIABLEORFUNCTIONCALL+
%OD%OA++++++%7Cintlit+%OD%OA++++++%7Cfloatlit+%OD%OA++++++
%7Clpar+ARITHEXPR+rpar+%OD%OA++++++%7Cnot+FACTOR+%OD%OA++++++
%7CSIGN+FACTOR+. %OD%OA%OD%OAFACTOR2+-%3E+lpar+APARAMS+rpar+%7C+REPTIDNEST1+
+. %OD%OA%OD%OA%OD%OA%OD%OAFAPARAMS+-%3E+id+colon+TYPE+REPTFPARAMS3+
REPTFPARAMS4++%OD%OA++++++%7C. %OD%OA%OD%OAFAPARAMSTAIL+-%3E+comma+
id+colon+TYPE+REPTFPARAMSTAIL4+. %OD%OA%OD%OAFUNCBODY+-%3E+lcurbr+
REPTFUNCBODY1+rcurbr+. %OD%OA%OD%OAFUNCDECL+-%3E+FUNCHEAD+semi+. %OD%
OA%OD%OAFUNCDEF+-%3E+FUNCHEAD+FUNCBODY+. %OD%OA%OD%OAFUNCHEAD+-%3E+
func+id+lpar+FPARAMS+rpar+arrow+RETURNTYPE+. ++%OD%OA%OD%OA%OD%OAAREPTVARIABLEORFUNCTIONCALL+
-%3E+IDNEST+REPTVARIABLEORFUNCTIONCALL+%7C+. %OD%OAIDNEST+-%3E+dot+
id+IDNEST2+. ++%OD%OAIDNEST2+-%3Elpar+APARAMS+rpar+%7C+REPTIDNEST1+
. ++%OD%OA%OD%OA%OD%OA%OD%OAIMPLDEF+-%3E+impl+id+lcurbr+REPTIMPLDEF3+
rcurbr+. %OD%OA%OD%OAINDICE+-%3E+lsqbr+ARITHEXPR+rsqbr+. %OD%OA%OD%
OAMEMBERDECL+-%3E+FUNCDECL++%OD%OA++++++%7CVARDECL+. %OD%OA%
OD%OAMULTOP+-%3E+mult+%OD%OA++++++%7Cdiv+%OD%OA++++++%7Cand+
. %OD%OA%OD%OAOPTRSTRUCTDECL2+-%3E+inherits+id+REPTOPTSTRUCTDECL22++%
OD%OA++++++%7C. %OD%OA%OD%OAOPROG+-%3E+REPTPROG+. %OD%OA%
OD%OARELEXPR+-%3E+ARITHEXPR+RELOP+ARITHEXPR+. %OD%OA%OD%OARELOP+-%3E+
eq++%OD%OA++++++%7Cneq+%OD%OA++++++%7Clt+%OD%OA++++++%7Cgt+%OD%
OA++++++%7Cleq+%OD%OA++++++%7Cgeq+. %OD%OA%OD%OAREPTAPARAMS1+-%
3E+APARAMSTAIL+REPTAPARAMS1+%OD%OA++++++%7C. %OD%OA%OD%OAREPTFPARAMS3+
-%3E+ARRAYSIZE+REPTFPARAMS3+%OD%OA++++++%7C. %OD%OA%OD%OAREPTFPARAMS4+
-%3E+FPARAMSTAIL+REPTFPARAMS4+%OD%OA++++++%7C. %OD%OA%OD%OAREPTFPARAMSTAIL4+
-%3E+ARRAYSIZE+REPTFPARAMSTAIL4++%OD%OA++++++%7C. %OD%
OA%OD%OAREPTFUNCBODY1+-%3E+VARDECLORSTAT+REPTFUNCBODY1+%OD%OA++++
++++++%7C. %OD%OA%OD%OA%OD%OAAREPTIDNEST1+-%3E+INDICE+REPTIDNEST1+
%OD%OA++++++%7C. %OD%OA%OD%OAAREPTIMPLDEF3+-%3E+FUNCDEF+REPTIMPLDEF3+
%OD%OA++++++%7C. %OD%OA%OD%OAAREPTOPTSTRUCTDECL22+-%3E+comma+
id+REPTOPTSTRUCTDECL22+%OD%OA++++++%7C. %OD%OA%OD%OAREPTPROG+
-%3E+STRUCTORIMPLORFUNC+REPTPROG++%OD%OA++++++%7C. %OD%OA%OD%
OAREPTSTATBLOCK1+-%3E+STATEMENT+REPTSTATBLOCK1++%OD%OA++++++
++++%7C. %OD%OA%OD%OAREPTSTRUCTDECL4+-%3E+VISIBILITY+MEMBERDECL+REPTSTRUCTDECL4+
%OD%OA++++++%7C. %OD%OA%OD%OAREPTVARDECL4+-%3E+ARRAYSIZE+
```

```

REPTVARDECL4+%OD%OA+++++++%7C.%OD%OA%OD%OA%OD%OA%OD%OARETURNType+
-%3E+TYPE++%OD%OA+++++++%7Cvoid+.%OD%OA%OD%OARIGHTRECARITHEXPR+
-%3E+%OD%OA+++++++%7CADDOP+TERM+RIGHTRECARITHEXPR+.+%OD%
OA%OD%OARIGHTRECTERM+-%3E+%OD%OA+++++++%7C+MULTOP+FACTOR+RIGHTRECTERM+
.%OD%OA%OD%OASIGN+-%3E+plus+%OD%OA+++++++%7Cminus+.+%OD%OA%OD%OASTATBLOCK+
-%3E+lcurbr+REPTSTATBLOCK1+rcurbr+%OD%OA+++++++%7CSTATEMENT+%OD%
OA+++++++%7C. +%OD%OA%OD%OASTATEMENT+-%3E+id+STATEMENTBODY++%OD%
OA+++++++%7Cif+lpar+RELEXPR+rpar+then+STATBLOCK+else+STATBLOCK+
semi+%OD%OA+++++++%7Cwhile+lpar+RELEXPR+rpar+STATBLOCK+semi+%OD%
OA+++++++%7Cread+lpar+VARIABLE+rpar+semi+%OD%OA+++++++%7Cwrite+
lpar+EXPR+rpar+semi+%OD%OA+++++++%7Creturn+lpar+EXPR+rpar+semi+
.%OD%OA%OD%OA%OD%OASTATEMENTBODY+-%3E+REPTIDNEST1+CHECKFORENDVAR+%OD%
OA+++++++%7C+lpar+APARAMS+rpar+CHECKFORENDFUNC+. %OD%OA%OD%
OAVARFUNCSTATENEST+-%3E+dot+id+VARFUNCSTATENEST2+. %OD%OA%OD%OAVARFUNCSTATENEST2+
-%3E+lpar+APARAMS+rpar+CHECKFORENDFUNC+%OD%OA+++++++%
7C+REPTIDNEST1+CHECKFORENDVAR+. %OD%OA%OD%OA%OD%OA%OD%OACHECKFORENDFUNC+
-%3E+semi++%OD%OA+++++++%7C+VARFUNCSTATENEST+. %OD%OA%OD%OACHECKFORENDVAR+
-%3E+ASSIGNOP+EXPR+semi+%OD%OA+++++++%7C+VARFUNCSTATENEST+
.%OD%OA%OD%OA%OD%OASTRUCTDECL+-%3E+struct+id+OPTSTRUCTDECL2+lcurbr+
REPTSTRUCTDECL4+rcurbr+semi+.+%OD%OA%OD%OASTRUCTORIMPLORFUNC+-%3E+
STRUCTDECL+%OD%OA+++++++%7CIMPLDEF+%OD%OA+++++++
+++++++%7CFUNCDEF+. %OD%OA%OD%OATERM+-%3E+FACTOR+RIGHTRECTERM+. %
OD%OA%OD%OATYPE+-%3E+integer+%OD%OA+++++++%7Cfloat+%OD%OA+++++++%7Cid+
.%OD%OA%OD%OAVARDECL+-%3E+let+id+colon+TYPE+REPTVARDECL4+semi+. %OD%
OA%OD%OAVARDECLORSTAT+-%3E+VARDECL++%OD%OA+++++++%7CSTATEMENT+
.%OD%OA%OD%OAVARIABLE+-%3E+id+VARIABLE2+. %OD%OA%OD%OA%OD%OAVARIABLE2+
-%3E++REPTIDNEST1+REPTVARIABLE++++. %OD%OAVARIABLE2+-%3E++lpar+APARAMS+
rpar+VARIDNEST++. %OD%OA%OD%OA%OD%OAREPTVARIABLE+-%3E++VARIDNEST+REPTVARIABLE+
++.%OD%OAREPTVARIABLE+-%3E++. %OD%OA%OD%OAVARIDNEST+-%3E+dot+id+VARIDNEST2+
.%OD%OA%OD%OAVARIDNEST2+-%3E+lpar+APARAMS+rpar+VARIDNEST++. %OD%OAVARIDNEST2+
-%3E+REPTIDNEST1++. %OD%OA%OD%OA%OD%OAVISIBILITY+-%3E+public+%OD%OA+
+++++++%7Cprivate+.

```